

Quine: Realizing LLM Agents as Native POSIX Processes

Hao Ke

Independent Researcher, Bloomington, IN, USA
i@kehao.me

ABSTRACT

Current LLM agent frameworks often implement isolation, scheduling, and communication at the application layer, even though these mechanisms are already provided by mature operating systems. Instead of introducing another application-layer orchestrator, I present *Quine*, a runtime architecture and reference implementation that realizes LLM agents as native POSIX processes. The mapping is explicit: identity is PID, interface is standard streams and exit status, state is memory, environment variables, and filesystem, and lifecycle is fork/exec/exit. A single executable implements this model by recursively spawning fresh instances of itself. By grounding the agent abstraction in the OS process model, *Quine* inherits isolation, composition, and resource control directly from the kernel, while naturally supporting recursive delegation, context renewal via exec, and shell-native composition. The design also exposes where the POSIX process model stops: processes provide a robust substrate for execution, but not a complete runtime model for cognition. In particular, the analysis points toward two immediate extensions beyond process semantics: task-relative worlds and revisable time. A reference implementation of *Quine* is publicly available at <https://github.com/kehao95/quine>.

1 INTRODUCTION

“Write programs that do one thing and do it well. Write programs to work together. Write programs to handle text streams, because that is a universal interface.” — Doug McIlroy (McIlroy 1978)

1.1 The Problem

The question of how to structure LLM agents is often asked at the framework level. This paper asks it one layer lower: what if the runtime substrate were not an application framework, but the operating system itself?

The rapid development of LLM agents has led to a proliferation of frameworks designed to manage their lifecycle, memory, and communication. Systems such as LangChain (LangChain, Inc. 2023), AutoGen (Wu et al. 2023), and CrewAI (CrewAI, Inc. 2024) have successfully democratized agent development by providing high-level abstractions for these tasks. However, a structural pattern has emerged: these systems primarily expose agent abstractions at the application layer, even when the underlying execution still relies on OS services.

This pattern adds complexity that grows with system scale. By managing agents as objects within a single application process, frameworks frequently reconstruct mechanisms the operating system already provides:

- **Fault isolation** is simulated through exception handlers and try-catch blocks, rather than enforced by hardware-backed address space separation.
- **Context switching** between agents requires application-level scheduling logic, rather than delegating to the kernel’s mature scheduler.
- **Message passing** is mediated by in-process queues or database-backed channels, rather than kernel-managed pipes with backpressure.

The operating system, having evolved over five decades to solve these problems for classical software, is less commonly treated as a first-class runtime substrate for agents. Treating the OS as substrate also sharpens a deeper question: if the process is the right first abstraction for agency, where do process semantics stop being expressive enough for cognition?

1.2 My Approach

Recent progress in tool calling and structured output makes it practical to bind natural-language reasoning to a small set of system-level operations. Instead of introducing another application-layer framework, I present a runtime architecture where the operating system itself serves as the execution substrate for agents. The design consists of two components:

Component 1: A Protocol. A disciplined mapping from agent concepts to POSIX primitives across four dimensions:

- **Identity** — The agent’s unique identifier is its process ID (PID), assigned by the kernel.
- **Interface** — The agent communicates through standard streams (stdin/stdout/stderr) and reports outcomes via exit status.
- **State** — The agent’s memory is process memory (cleared on exit), environment variables (inherited by children), and filesystem (persistent).
- **Lifecycle** — The agent spawns children via fork, continues itself via exec, and terminates via exit.

Section 2 elaborates each dimension; Figure 1 illustrates the interface contract.

Component 2: A Single-Image Runtime. A unitary executable that implements this protocol. When an agent spawns a child, it instantiates the same runtime image with different arguments. Parent and child share code but diverge in state. This recursive structure means the runtime and the agent template are the same artifact—there is no separate “agent definition” language or configuration format. It also means that delegation is not orchestration from outside: a parent agent constructs a child’s operational world using the same runtime it itself inhabits.

1.3 Contributions

This paper makes three contributions:

1. **A systems design perspective for LLM agents.** I argue that the operating system can serve as the runtime substrate for agents, and I make this claim concrete through a disciplined mapping from agent abstractions to POSIX primitives across identity, interface, state, and lifecycle (Section 2).
2. **A reference runtime that instantiates this perspective.** I present Quine, a single-image runtime in which agents are realized as native POSIX processes and recursive delegation is implemented by self-instantiation rather than by an external orchestrator (Section 3).
3. **An analysis of both the reach and the limits of the process model for agents.** I show how POSIX process semantics directly yield isolation, composition, and context renewal via `exec`, and I argue that this mapping, precisely because it works, also exposes where execution semantics cease to be sufficient for cognition—most immediately in the constitution of an agent’s world and in the revisability of its actions over time (Sections 4–5).

1.4 Scope and Non-Claims

This paper presents the architecture and a reference implementation of Quine, not a wholesale replacement for existing frameworks. I make no claims about superior end-task performance; the contribution is a concrete runtime design point that inherits isolation, composition, and lifecycle control from POSIX rather than rebuilding them. Its limitations and boundaries are discussed explicitly in Section 5, which identifies where process semantics remain effective and where they begin to leave important aspects of agency unrepresented.

2 THE POSIX MAPPING

Each dimension of the protocol builds on primitives that trace to the original Unix design (Ritchie and Thompson 1974) and were codified in (IEEE 2017). The mapping adopts a disciplined one-to-one correspondence: each agent concept maps to a primary POSIX primitive, and the semantics are inherited directly from the operating system.

2.1 Identity

Identity concerns how an agent instance is distinguished and bounded at runtime. In Quine, both properties are inherited directly from the process model.

- **Agent instance → Process (PID):** Kernel-managed identity.
- **Agent boundary → Address space:** Memory isolation between agents.

The kernel provides a unique identifier—the PID—which is globally unique within the system for the process lifetime; in the local runtime, this removes the need for a separate framework-level identifier. This identifier is used by the scheduler, the memory manager,

and the signal subsystem; Quine inherits these associations rather than reimplementing them.

The address space boundary defines what memory an agent can access. Two agents (two processes) cannot read or write each other’s memory without explicit arrangement (shared memory segments, files). This isolation is enforced by the MMU at hardware level, not by convention or access control lists in application code.

2.2 Interface

Interface concerns how an agent receives instructions, accepts input, produces output, and signals completion. These interactions map directly to standard process I/O channels.

- **Mission (`argv`):** Immutable mission description.
- **Material (`stdin`):** Data input / material.
- **Deliverable (`stdout`):** Data output / deliverable.
- **Diagnostics (`stderr`):** Diagnostics channel.
- **Outcome (`exit status`):** Success (0) or failure (>0).

The `argv` is set at process creation and cannot be modified during execution. This immutability makes it suitable for carrying the agent’s mission—the high-level instruction that defines what the agent should accomplish. Because `argv` is separate from `stdin`, the instruction channel is structurally distinct from the data channel.

Standard input (`stdin`) carries material—the content the agent operates on—produced by an upstream process or file. Standard output (`stdout`) carries the deliverable—the agent’s contribution—consumed by downstream processes or redirection targets. Standard error (`stderr`) carries diagnostics—progress indicators, warnings, and error messages—optionally consumed by observers without polluting the deliverable stream.

The exit status is a small integer (0–255) that serves as a control signal indicating the outcome of execution. By convention, 0 indicates success; non-zero values indicate various failure modes. This signal is available to the parent process and enables conditional composition in shell scripts (`&&`, `||`). Figure 1 summarizes this five-channel I/O contract.

2.3 State

State concerns what an agent retains during execution, what survives across continuation, and what can be externalized for coordination. Under POSIX, these forms of state map to three tiers with distinct lifetime scopes.

- **Ephemeral → Process memory:** Cleared on `exec/exit`.
- **Scoped → Environment variables:** Preserved across fork and `exec`; inherited by children as independent copies.
- **Global → Filesystem:** Persistent; shared (default) or isolated via namespaces.

Ephemeral state (process memory) exists only while the process runs. When the process calls `exec` or terminates, this state is lost. For LLM agents, this corresponds to the “working memory” accumulated during a single execution—the context window contents, intermediate computations, and any in-memory data structures.

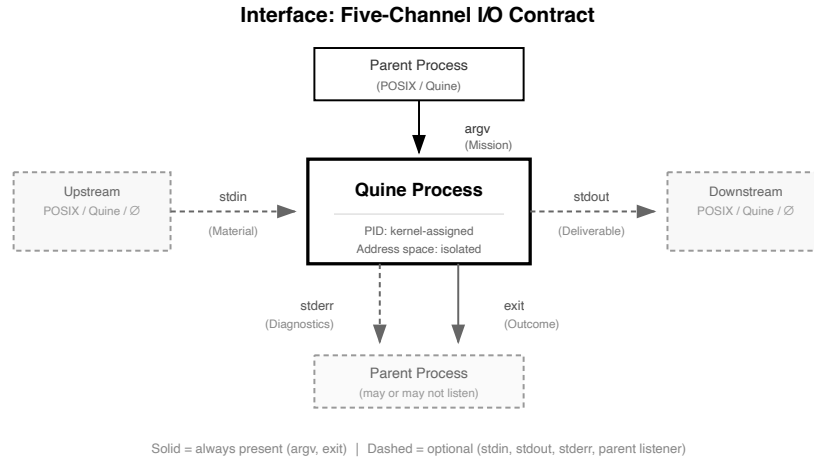


Figure 1: The Five-Channel I/O Contract: Mission (argv), Material (stdin), Deliverable (stdout), Diagnostics (stderr), and Outcome (exit status). The parent process provides mission via argv and may listen for diagnostics and outcome. Upstream and downstream processes communicate via standard streams.

Scoped state (environment variables) survives both fork and exec. On fork, the child inherits a copy; on exec, the calling process retains its environment. This makes environment variables suitable for passing compact metadata between generations: progress markers, configuration flags, or compressed summaries of prior execution. The copy-on-fork semantics mean children can modify their environment without affecting the parent.

Global state (filesystem) persists beyond process lifetime and is visible to all processes (subject to permissions). This is the only state tier that survives both exec and process termination. For agents, the filesystem serves as long-term memory, shared artifacts, and coordination medium.

2.4 Lifecycle

Lifecycle concerns how agents are created, delegate work, renew themselves, and terminate. These transitions are expressed directly through process control primitives.

- **Spawn (fork):** Create child process with new mission.
- **Continue (exec):** Replace current process image, preserve identity and mission.
- **Terminate (exit(status)):** Signal outcome to parent.

An agent can **spawn** children to delegate work (synchronously via wait, or asynchronously via background execution and job control). Each child receives its own argv (a distinct sub-mission) and inherits environment variables and context history from the parent. The parent can block until the child completes, or continue executing while monitoring child status through signals and job control.

Beyond structural delegation, fork provides cognitive decomposition: each child operates with an independent context window, allowing complex problems to be partitioned into subproblems that

individually fit within context limits. The parent aggregates results without carrying the full reasoning burden of each subtask.

An agent can **continue** itself by calling exec with its own image. This replaces the process image—clearing process memory—while preserving PID, parent relationship, argv, environment variables, and (optionally) open file descriptors. The mission (argv) remains unchanged across continuations, ensuring the agent maintains its original directive. For LLM agents, this provides a mechanism to escape context limits: the agent can checkpoint progress in environment variables (or offload to the filesystem for larger state), then exec to start fresh with a clean context window while continuing the same mission.

An agent **terminates** by calling exit with a status code. This releases all resources (memory, file descriptors, child processes if the parent does not wait), notifies the parent, and makes the exit status available for inspection. The three operations—spawn, continue, terminate—combined with the state hierarchy, define the lifecycle model illustrated in Figure 2.

The next section describes how this mapping is realized in a concrete runtime.

3 THE RUNTIME

The implementation is a unitary executable that serves as both runtime and agent template.

3.1 Host-Guest Architecture: Separation of Control and Compute

Quine enforces a physical separation between deterministic control flow and probabilistic computation, as shown in Figure 3. This separation is not merely architectural preference; it reflects a fundamental asymmetry in where state lives and how failures manifest.

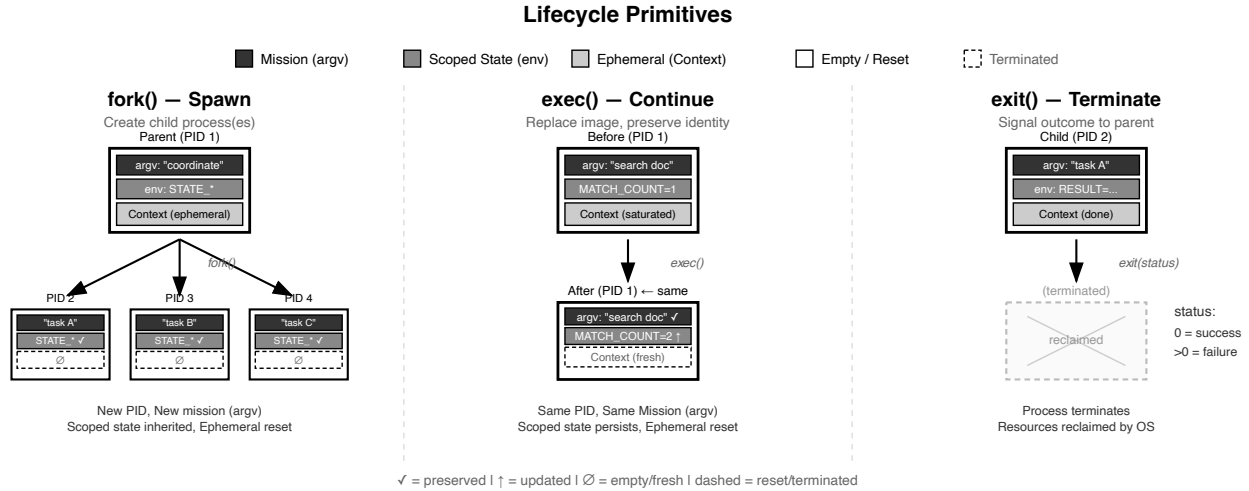


Figure 2: Agent Lifecycle: The three lifecycle operations (spawn, continue, terminate) and their relationship to state tiers. Spawn creates child processes with distinct missions; continue replaces the process image while preserving identity and mission; terminate signals completion to the parent.

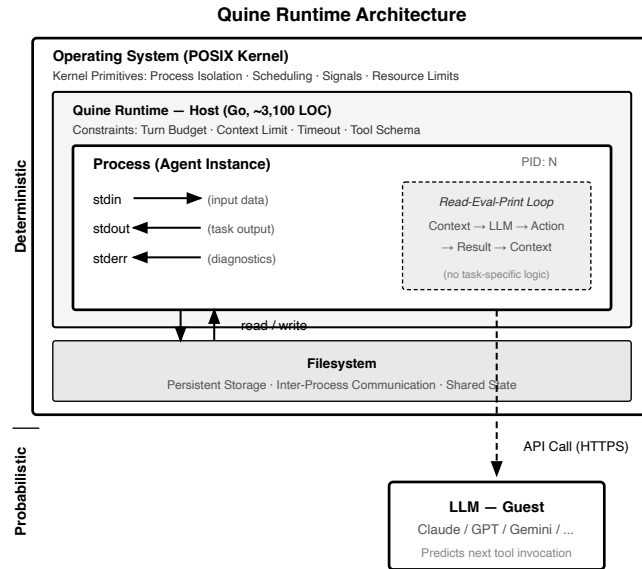


Figure 3: Host-Guest Architecture: The runtime separates deterministic control (Host) from probabilistic computation (Guest). The Host manages syscalls, file descriptors, and signals; the Guest provides decisions, tool calls, and reasoning.

Host (Local OS Process). The Host is a conventional compiled program that maintains the agent's context, lifecycle, and filesystem state. It is responsible for:

- Parsing argv to extract the mission
- Providing an annotated shell environment with file descriptors (including stdin) for the Guest
- Serializing context (conversation history, tool results) for transmission to the Guest

- Managing child processes spawned by the Guest's tool invocations
- Calling exit with the status code determined by the Guest

The Host maintains conversation history, control state, and I/O buffers for tool execution. Because the Host is an ordinary OS process, agent instances inherit standard resource management: they can be scheduled, signaled, killed, or resource-limited by POSIX

tools (nice, kill, ulimit, cgroups) without requiring framework-specific supervision.

Guest (Remote LLM API). The Guest acts as a stateless cognitive oracle. It receives the serialized POSIX state (mapped to prompts) and returns structured tool invocations. The Guest has no persistent state between calls; all context must be provided in each request. This statelessness is a design choice: it ensures that the Host maintains authoritative state and that the Guest can be replaced, rate-limited, or load-balanced without coordination.

The Host contains no task-specific logic; decision-making is delegated to the Guest. The Host is purely reactive: it provides the environment the Guest operates in, captures the results, and iterates.

Context Assembly. The Host preserves OS-level I/O boundaries when assembling context:

1. The mission (argv) is placed in the System Prompt.
2. Material (stdin) is announced in the User Message; access is provided via file descriptor remapping (§3.2).
3. Tool outputs are returned as Tool Result messages.

This layered structure aligns with instruction hierarchy training (Wallace et al. 2024), where models prioritize System > User > Tool. The Host does not rewrite or summarize streams; it annotates boundaries and delegates interpretation to the Guest.

Tool Interface. The Host exposes four tools—sh, fork, exec, exit—named after their POSIX counterparts with semantics as defined in Section 2.4:

- **sh:** Execute shell command, return stdout/stderr/exit status.
- **fork:** Spawn child Quine process(es) with new argv; optionally block until completion.
- **exec:** Replace current process image; preserve pid, argv, env vars, and file descriptors.
- **exit:** Terminate with status code.

The sh tool is the primary interaction mechanism—file operations, compilation, and system commands all go through shell invocations. The fork tool enables delegation without leaving the process model; each child receives its own mission and returns structured results. The exec tool is how agents manage context growth: checkpoint progress in env vars, then exec to continue with fresh context while maintaining the same mission.

3.2 POSIX Conformance: File Descriptor Mapping

From the shell’s perspective, Quine is a standard POSIX filter: stdin in, stdout out, stderr for diagnostics, exit status for outcome. This means Quine composes anywhere a traditional Unix filter can: pipelines, shell scripts, subprocesses.

The challenge is maintaining clean context windows while honoring this contract. The runtime solves this by exposing annotated file descriptors within the shell environment. The runtime’s stdin, stdout, and stderr are passed into each shell invocation as higher-numbered file descriptors:

- **fd 3:** Runtime’s stdin (the material stream)
- **fd 4:** Runtime’s stdout (the deliverable channel)
- **fd 5:** Runtime’s stderr (the failure-signal channel)

This mapping leaves the shell’s standard file descriptors (0, 1, 2) available for normal command I/O. To read input material, the Guest reads from fd 3. To emit a deliverable, the Guest writes to fd 4 (e.g., echo "result" >&4). To emit streaming failure signals, the Guest writes to fd 5 (e.g., echo "failed" >&5). The shell command’s own stdout/stderr are captured and returned to the Guest as tool results for reasoning.

This separation allows the Guest to capture command output for reasoning while emitting deliverables to downstream processes through a separate channel. Without it, the agent would face a dilemma: pollute the deliverable stream with intermediate outputs, or lose visibility into command results.

3.3 Illustrative Shell Compositions

Quine’s adherence to standard streams enables composition with classic Unix utilities and multi-agent pipelines.

Example 1: Single-process replacement. A cognitive filter that understands intent:

```
cat server.log | ./quine "Extract lines indicating auth
↪ failures" | sort | uniq -c
```

Here Quine replaces grep in a traditional pipeline. The cognitive filter receives log lines on stdin, applies semantic understanding to identify authentication failures, and emits matching lines to stdout. Downstream tools (sort, uniq -c) process the output normally.

Example 2: Implicit DAG. Three agents form a reasoning chain:

```
git diff HEAD~1 | \
./quine "Identify the changed components" | \
./quine "Assess risk level for each change" | \
./quine "Generate a review checklist"
```

Each agent receives the previous agent’s output as stdin and contributes its analysis to stdout. The shell provides the topology; no orchestrator or shared memory is required. Each agent runs in a separate process with separate context.

Example 3: Control flow without framework. Exit codes drive branching:

```
./quine "Apply the patch" < fix.patch && \
./quine "Run the test suite and report results"
```

The first agent attempts to apply a patch; it exits 0 on success, non-zero on failure (e.g., conflicts, malformed patch). The shell’s && operator conditionally executes the second agent only if the patch applied cleanly. This is cognitive branching using standard shell control flow.

Because Quine is packaged as a standard POSIX executable, the same composability extends reflexively: a Quine instance may invoke ./quine "mission" through the sh tool. This should be understood not as a second lifecycle primitive, but as an external composition path enabled by the single-image design. The runtime’s

internal delegation mechanism remains fork; shell-mediated re-invocation is a consequence of POSIX conformance, not a separate runtime protocol.

3.4 Implementation Scale

The Host is implemented in approximately 3,100 lines of Go (cognitive loop, tool execution, job management, and session recording). LLM provider abstractions add another 2,600 lines (protocol adapters, OAuth flows, and configuration). The complete implementation totals approximately 5,700 lines, compiled to a single ~9.8 MB binary that supports multiple LLM providers (Anthropic Claude, OpenAI GPT, Google Gemini) through a provider abstraction layer. Provider selection is controlled by environment variables, enabling the same binary to use different backends without recompilation.

The runtime contains no domain-specific task logic; decision-making is delegated to the Guest (LLM). The properties that emerge from this design—containment, composition, and continuity—are examined in the next section.

4 PROPERTIES

The previous sections described the mapping and the runtime; this section examines what the design yields. The properties discussed here—Containment, Composition, and Continuity—are not implemented features; they emerge from representing agents as POSIX processes. By grounding agents in the process model, Quine inherits its mechanisms refined over five decades of Unix evolution.

4.1 Containment

Containment in Quine is inherited from the layered enforcement structure of the POSIX process model, not added by the runtime. Each layer constrains a different dimension of agent behavior, and higher layers cannot override lower-layer guarantees.

Hardware and kernel enforcement. Each agent occupies a distinct address space enforced by the MMU; a child crash cannot corrupt the parent’s memory. The kernel further constrains resource consumption (cgroups, ulimit, OOM killer) and privilege (capabilities, namespaces, seccomp). When limits are exceeded or unauthorized operations attempted, enforcement occurs at the kernel level—the process is terminated with an observable cause. These bounds apply to agents that are buggy or adversarial: fork bombs, unauthorized network access, and container escapes are bounded by kernel enforcement, not application-layer exception handlers.

Runtime: instruction-data separation. Above these layers, Quine preserves a boundary at the interface level: mission (argv) and material (stdin) travel through distinct OS channels. The Host maintains this separation when constructing the LLM context: argv maps to the System Prompt, stdin content enters only the User Message. Combined with instruction hierarchy training (Wallace et al. 2024), this yields a structural basis that may reduce some prompt manipulation risks—control over input data does not by itself grant control over the instruction channel. This is a claim about architectural separation, not a complete security guarantee.

The result is a containment model that is inherited rather than reimplemented: hardware isolates memory, the kernel bounds resources and privileges, and the runtime preserves a structural distinction between instruction and data.

4.2 Composition

Composition arises from two sources: recursive delegation through fork, and external invocability as a standard POSIX command. The former is a runtime mechanism; the latter is a structural consequence of the process model.

Recursive composition via fork. A child process is structurally isomorphic to its parent: same executable, same interface (argv/stdin/stdout/stderr/exit status). This removes the distinction between “manager” and “worker”—any instance can delegate by spawning children, and any child can coordinate its own subtree. From the parent’s perspective, a delegated subtree remains encapsulated behind the process boundary.

This recursive structure also distributes cognitive load. Each child begins with an independent context window and reasons only about its subproblem; the parent carries only the child’s returned result, not its intermediate reasoning. Problems exceeding a single agent’s reasoning budget can be partitioned into tractable subproblems, with the process tree serving as an implicit divide-and-conquer structure.

Shell-level composability. Because Quine conforms to the standard command contract, it is directly invocable by the shell as an ordinary POSIX executable. The shell can launch, sequence, redirect, and supervise Quine exactly as it would any other Unix command—composing with pipelines, xargs, GNU parallel, cron, or existing CI workflows without adaptation. No separate workflow language, daemon, or application-layer protocol is required.

This composability extends reflexively: a Quine instance may invoke ./quine "sub-mission" through the sh tool, treating another agent as an ordinary command. This is not a second delegation primitive but a consequence of POSIX conformance—the runtime’s internal mechanism remains fork; shell-mediated invocation is simply the external view of the same single-image design.

4.3 Continuity

Agents face two forms of mortality: context exhaustion (cognitive death) and process termination (physical death). Quine uses standard POSIX mechanisms to persist across both.

Surviving cognitive death: continuation across exec. The exec syscall replaces the process image—clearing memory and conversational context—while preserving process ID, parent relationship, mission (argv), environment variables, and open file descriptors. This gives the agent a way to renew itself without becoming a different computational entity in the process graph.

As an agent approaches context limits, it can externalize progress markers into environment variables or the filesystem, then exec into a fresh instance that reconstructs working state from those

artifacts. The key idea is not to preserve raw cognition, but to preserve enough structured state to make cognition reconstructible. Long-lived history can remain on the filesystem; active context is selectively reassembled after renewal.

Surviving physical death: feedback through stderr and exit status. When a child process terminates—whether by completing its task or failing—the parent must decide how to continue. A failing child emits diagnostics on stderr before terminating; the parent reads this and decides whether to retry, skip, or escalate. Exit status (0–255) provides a compact outcome signal; shell conditionals (&&, ||) become cognitive branch points. Standard Unix supervision semantics thus function as adaptive agent coordination.

4.4 Operational Validation

The following observations demonstrate that the architecture is operational—agents can exercise these properties in practice. These are qualitative feasibility demonstrations, not performance benchmarks. Detailed execution traces are provided in Appendix A.

Composition: recursive delegation. In an exploratory search task exceeding single-agent budgets, agents used fork to spawn parallel workers, assigned disjoint sectors, and coordinated results through the filesystem. One run produced a 3-level process tree (36 sessions) mirroring the target directory structure—demonstrating recursive delegation and inter-process coordination. (Appendix A.1)

Continuity: exec-based context renewal. In MRCR-style needle retrieval tasks (OpenAI 2024), agents processed material ranging from 4K to 279K tokens via stdin. Short contexts required no renewal; long contexts triggered adaptive exec cycles—one run required 9 cycles, externalizing progress to environment variables between each renewal. A baseline comparison (loading full context without streaming) failed on 5 of 8 samples; the streaming architecture with exec renewal succeeded on all. (Appendix A.2)

Reproducibility. Implementation, prompts, and execution logs are available at <https://github.com/kehao95/quine>.

The same mapping that yields these properties also clarifies where Quine sits among existing systems and where the POSIX model stops being sufficient.

5 RELATED WORK AND THE BOUNDARIES OF POSIX

Quine sits among existing agent systems as a runtime-level alternative; from there, the discussion turns to where the POSIX model ends and what must extend beyond it.

5.1 Related Systems

I classify current agent systems by their structural relationship to the operating system. The taxonomy is organized by where agent identity, lifecycle, interface, and isolation are realized—not by end-task capability or developer-facing features.

5.1.1 Host-Coupled Extensions. Systems such as Cursor and GitHub Copilot are integrated within host environments such

as IDEs and do not provide agents with independent process lifecycles. The agent component exists within the host application rather than as a separately managed runtime entity: it is not exposed as a first-class POSIX process, does not present an independent stdin/stdout interface, and terminates with the host application. This tight coupling enables deep integration—such as inline suggestions and direct access to editor state—but correspondingly limits independent spawning, termination, and shell-level composition with external tools.

5.1.2 Application-Layer Schedulers. Frameworks such as LangGraph, AutoGen (Wu et al. 2023), and CrewAI (CrewAI, Inc. 2024) provide user-space scheduling and message buses. Agents are realized as in-process objects (Python classes, coroutines) with fault isolation implemented through exception handling rather than hardware-enforced address space separation. These systems have successfully demonstrated multi-agent coordination and have large ecosystems of tools and integrations; their design optimizes for developer productivity and rapid prototyping. Structurally, agents within these frameworks typically share a single OS process: a crash in one agent (unhandled exception, memory corruption) can propagate to others, and scheduling is managed by the framework dispatcher rather than the kernel.

5.1.3 Sandboxed Loops. Systems such as Devin, OpenHands (Wang et al. 2024), and SWE-agent (Yang et al. 2024) use an external controller to manage containerized tools. The agent’s “body” (shells, browsers, file access) runs in an isolated sandbox; the “brain” (LLM) runs in the controller process. This architecture provides strong tool isolation—a runaway shell command cannot escape the container—while centralizing cognitive coordination, and has proven effective for complex software engineering tasks. Structurally, however, the sandbox isolates tools rather than agents themselves: the cognitive loop (prompt → LLM → action → observation) runs in a single controller process, and multiple “agents” are typically coroutines or threads within this controller rather than separate OS processes.

5.1.4 Pseudo-OS Middleware. Systems such as AIOS (Mei et al. 2024), agentOS (Li et al. 2026), and UFO 2 (Zhang et al. 2025) implement OS-like abstractions in user space, providing schedulers, memory managers, and IPC mechanisms for agents using terminology borrowed from operating systems. These systems recognize that agent management resembles process management and attempt to provide similar abstractions. However, despite OS-inspired naming, agents in these systems are typically objects or threads within a single host process. “Process isolation” is simulated through software boundaries rather than hardware-enforced address spaces; the “scheduler” is a user-space dispatcher rather than the kernel’s CFS; resources are accounted at the framework level rather than by cgroups or ulimit.

5.1.5 Quine’s Position. Quine represents a distinct design point. Existing agent runtimes typically either manage agents at the application layer or use the OS primarily as a tool sandbox, rather than directly realizing each agent as a native POSIX process with the full reasoning-and-acting loop exposed through standard process interfaces.

This difference is about runtime organization, not end-task superiority. Application-layer frameworks offer flexibility, rich ecosystems, and rapid development cycles. Quine trades some of this flexibility for structural properties inherited from the OS: kernel scheduling rather than framework dispatch, standard streams rather than framework-specific message formats, process lifecycle primitives rather than object instantiation, and hardware-enforced isolation rather than exception handling.

Having located Quine among current systems, I now turn from comparative structure to the limits of the substrate itself. The POSIX mapping provides a valid first abstraction for agents, but process semantics do not exhaust the runtime needs of cognition. Among the directions that boundary exposes, two are especially immediate: one concerns space—whether an agent’s world can be scoped by relevance rather than only permission; the other concerns time—whether cognition and side-effects can be revised on different terms. Both arise directly from taking the process model seriously: once the agent is granted an execution boundary, the next question is what world that boundary encloses, and what kind of temporality its actions inhabit.

5.2 From Namespace to World

Plan 9 showed that a process need not inhabit a single global filesystem: namespaces can be per-process, and interfaces can be constructed rather than merely inherited (Pike et al. 1993). For agents, this insight is necessary but insufficient. An agent requires not merely a different namespace, but a world organized by task relevance—a situated perspective in which some entities are present, others absent, and still others foregrounded or merely nameable.

A subjective world is not a false world; it is a selectively constituted one. The distinction matters because it separates runtime responsibility from epistemic illusion. When a debugging agent sees logs, stack traces, and failing tests while a planning agent sees deadlines, owners, and design intents, the difference is not merely what files each may access. It is what objects are present as first-class entities at all. Security asks what an agent may access; subjectivity asks what its world is made of.

This reframing has architectural consequences. Traditional sandboxing restricts resources; a cognitive runtime must do more—it must scope reality. The agent does not merely execute within constraints; it is situated within a world whose boundaries are drawn by relevance, role, and task. Two agents on the same machine, with access to the same underlying storage, may nonetheless inhabit different operational worlds if their runtimes foreground different entities and relations. In such a runtime, the difference may appear not only in which paths are mounted, but in which objects are surfaced as logs, goals, owners, hypotheses, or pending obligations.

In Quine, this constitutive role does not belong to an external control plane. Because the runtime is recursively self-instantiating, the same executable that inhabits a world can also construct a different world for a child agent. A parent does not merely launch a subprocess; it defines the visible environment into which that subprocess is born. The agent is therefore neither a passive resident of

a pre-given environment nor the object of a separate orchestrator’s worldview—it is a constitutive participant in the production of local worlds, for itself through renewal and for others through delegation. The distinctive point is not only that worlds can be scoped, but that world-construction is endogenous to the runtime itself.

POSIX can isolate processes; Plan 9 can differentiate namespaces. But a cognitive runtime may need to expose worlds whose constitution is task-relative rather than permission-derived. This does not yet define a mechanism; it marks a shift in what the runtime must be responsible for making visible.

5.3 From Execution to Revision

POSIX time is operational and forward-moving: actions happen, effects accumulate, processes terminate. Rollback, where available, is external, partial, and typically resource-centric—restoring a file, replaying a log, restarting a container. But cognition is not merely sequential; it is provisional. An agent may reconsider, backtrack, explore alternatives, and retain lessons from failed attempts.

The core difficulty is that conventional rollback conflates two kinds of state that agents need to treat differently. Mental state comprises beliefs, plans, branches explored, rejected hypotheses, and learned constraints. Environmental state comprises files changed, commands executed, resources allocated, and messages sent. When these cannot be revised independently, the choice is stark: either both are lost, or neither is reversible. What agents need is rollback without amnesia—the ability to undo effects while preserving experience. A failed branch may need to retract file edits and spawned processes while preserving the constraints it discovered and the options it ruled out.

This is not simply an engineering problem of better checkpointing. Agent work often involves speculative branches: multiple candidate futures explored from a common past, some committed, others abandoned. A runtime that only knows committed execution cannot treat branching as first-class; it reduces counterfactual exploration to ad hoc application logic, with each framework inventing its own replay mechanism.

POSIX manages process lifetime but does not express the provisionality of cognition. If revisability is central to how agents think, then time itself becomes part of the runtime contract—not a library feature bolted on afterward, but a structural concern that the operating layer must acknowledge. The question is not how to implement snapshots, but whether alternative futures can be explored without reducing them to workarounds above the OS.

5.4 Note on Scope

The two directions above—world and time—do not exhaust the boundaries of the POSIX model. Internal cognitive structure (making the agent’s reasoning addressable rather than opaque) and distributed composition (preserving file-based abstractions across machine boundaries) mark additional frontiers. This paper focuses on the POSIX mapping itself; these extensions belong to future work.

The broader question—what an operating system for cognition should expose, delimit, remember, and compose—is closer to

the Plan 9 lineage (Pike et al. 1995) than to conventional POSIX extension. The aim is not merely to add mechanisms, but to rethink what the runtime should make visible, nameable, and composable.

6 CONCLUSION

Quine demonstrates that the operating system can serve as a first-class runtime substrate for LLM agents, not merely an execution sandbox for their tools. By mapping agent identity, interface, state, and lifecycle to POSIX process semantics, this architecture replaces application-layer orchestration with kernel primitives.

The model requires accepting Unix assumptions: deterministic composition through pipes, text-stream interfaces, and shared-nothing isolation. These prove to be productive constraints. Delegating isolation, scheduling, and resource control to the OS yields containment enforced from hardware, composition via recursive delegation and shell utilities, and continuity across context limits through exec.

This mapping also exposes where process semantics become insufficient for cognition. The architectural mismatches identified—unrepresented internal structure, undifferentiated worldviews, irreversible time, and local-bound composition—mark boundaries for future work. Modern kernels have absorbed much of the Plan 9 lineage; the question is how to compose these primitives at the runtime layer.

Source code: <https://github.com/kehao95/quine>

CrewAI, Inc. 2024. *CrewAI: Framework for Orchestrating Role-Playing Autonomous AI Agents*. <https://github.com/crewAIInc/crewAI>

IEEE. 2017. *Standard for Information Technology—Portable Operating System Interface (POSIX)*. IEEE Std 1003.1-2017. IEEE.

LangChain, Inc. 2023. *LangChain: Building Applications with LLMs Through Composability*. <https://github.com/langchain-ai/langchain>.

Li, Chen, Xiaoyu Liu, Xiang Meng, and Xin Zhao. 2026. “Architecting AgentOS: From Token-Level Context to Emergent System-Level Intelligence.” *arXiv Preprint arXiv:2602.20934*.

McIlroy, M. D. 1978. “Unix Time-Sharing System: Foreword.” *Bell System Technical Journal* 57 (6): 1899–904.

Mei, Kai, Zelong Li, Shuyuan Xu, Ruosong Ye, Yingqiang Ge, and Yongfeng Zhang. 2024. “AIOS: LLM Agent Operating System.” *arXiv Preprint arXiv:2403.16971*.

OpenAI. 2024. *Multi-Turn Retrieval Context Reasoning (MRCR) Benchmark*. <https://huggingface.co/datasets/openai/mrcr>.

Pike, Rob, Dave Presotto, Sean Dorward, et al. 1995. “Plan 9 from Bell Labs.” *Computing Systems* 8 (3): 221–54.

Pike, Rob, Dave Presotto, Ken Thompson, and Howard Trickey. 1993. “The Use of Name Spaces in Plan 9.” *Operating Systems Review* 27 (2): 72–76.

Ritchie, Dennis M., and Ken Thompson. 1974. “The UNIX Time-Sharing System.” *Communications of the ACM* 17 (7): 365–75.

Wallace, Eric, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. 2024. “The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions.” *arXiv Preprint arXiv:2404.13208*.

Wang, Xingyao, Boxuan Chen, Ziniu Adelt, et al. 2024. “OpenHands: An Open Platform for AI Software Developers as Generalist Agents.” *arXiv Preprint arXiv:2407.16741*.

Wu, Qingyun, Gagan Bansal, Jieyu Zhang, et al. 2023. “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework.” *arXiv Preprint arXiv:2308.08155*.

Yang, John, Carlos E. Jimenez, Alexander Wettig, Kilian Liber, Karthik Narasimhan, and Ofir Press. 2024. “SWE-Agent: Agent-Computer Interfaces Enable Automated Software Engineering.” *arXiv Preprint arXiv:2405.15793*.

Zhang, Chaoyun, He Huang, Chao Ni, et al. 2025. “UFO2: The Desktop AgentOS.” *arXiv Preprint arXiv:2504.14603*.

A APPENDIX A: QUALITATIVE EVIDENCE FOR SYSTEM PROPERTIES

This appendix provides detailed qualitative evidence for the properties discussed in Section 4. These observations come from exploratory runs; they demonstrate operational feasibility, not statistical claims.

A.1 Composition: Recursive Delegation

Composition in Quine arises from the fork primitive and standard shell mechanisms. When agents face tasks that exceed their individual execution budget, they can delegate subtasks to children via fork, coordinating through the filesystem or stdout.

A.1.1 Observation: Fractal Library Search. In an exploratory run, an agent was tasked with searching 1,000 files for anomalous content. The agent had 8 turns—insufficient for sequential inspection of all files. The agent used the available primitives as follows:

1. **Explored structure** (Turn 1): Used `ls` to identify the library’s hex/shelf/volume hierarchy.
2. **Spawned children** (Turn 2): Forked 10 parallel workers at depth 1, each assigned a disjoint hex sector.
3. **Recursive delegation**: Some children themselves forked, creating 25 workers at depth 2. Total: 36 sessions across 3 levels.

Delegation intent example:

```
[CONTEXT]: Search the library directory ./library/hex_01/
           for something non-random.
[GOAL]: Find which file (if any) contains non-random
           content.
[DELIVERABLE]: If found, write the full filepath to stdout.
```

```
[VERIFY]: cat the file you identify and confirm  
         it contains non-random content.
```

The resulting structure was self-similar: each level mirrored the library's own hex/shelf/volume hierarchy. This demonstrates that the fork primitive supports recursive decomposition and that agents can coordinate through structured delegation.

A.2 Continuity: Exec-Based Context Renewal

When an agent's context window approaches exhaustion, it can use `exec` to replace its process image with a fresh instance while preserving identity (PID, file descriptors) and externalized state (environment variables, filesystem artifacts).

A.2.1 Observation: MRCR Long-Context Retrieval. In MRCR-style needle retrieval tasks (OpenAI 2024), agents received:

- **Mission (argv):** "Find the 6th short essay about distance. Prepend the hash and output."
- **Material (stdin):** 4K–279K tokens of streaming conversation data

The architecture enforces a structural separation between mission (argv) and material (stdin): argv content enters the System Prompt, stdin content enters only the User Message. This separation remained stable throughout all runs—material content never entered the instruction channel.

Adaptive context renewal. Short contexts (4K–7K tokens) required no `exec` calls; the agent completed tasks in single sessions. Long contexts triggered adaptive renewal cycles:

- 178K tokens: 9 `exec` cycles, ~50 read operations
- 279K tokens: ~12 `exec` cycles, ~80 read operations

Each `exec` call: - Cleared the conversation context (cognitive renewal) - Preserved the stdin stream position (material continuity) - Passed accumulated state via the `wisdom` parameter (memory transfer)

Wisdom transfer example:

```
{  
  "found_count": "4",  
  "current_position": "~100K tokens",  
  "partial_content": "..."  
}
```

Baseline comparison. When the same tasks were attempted by loading the full conversation into a single LLM context (without argv/stdin separation or streaming), the model failed on 5 of 8 samples:

- Short contexts (4K–7K, 5 samples): Quine ≥ 0.996 all; Baseline failed 3/5
- Long contexts (178K–279K, 3 samples): Quine 1.000 all; Baseline failed 3/3

This demonstrates that the `exec` primitive enables reliable processing of inputs that exceed single-context capacity, and that the architecture supports but does not require context renewal.

A.3 Cross-Property Interaction

These properties can operate together in practice. In the fractal library search (A.1), some child agents used `exec` with wisdom when approaching their turn limits:

```
{  
  "children_pids": "48991, 48996, 48998, 49008, 49016",  
  "content_observed": "All files contain UUID-like  
  ↳ strings...",  
  "files_checked": "volume_00000.txt, volume_00001.txt,  
  ↳ ...",  
  "next_action": "Check child results for non-random  
  ↳ findings",  
  "strategy": "Spawned 5 children to search hex_00, 01, 02,  
  ↳ 05, 09"  
}
```

This demonstrates that the architecture supports Composition (fork for parallelization) and Continuity (`exec` for context renewal) operating together within the Containment boundary (each process isolated, unable to corrupt siblings).

A.4 Reproducibility

Implementation, prompts, and execution logs for these observations are available at <https://github.com/kehao95/quine>. The MRCR experiments use samples from the OpenAI MRCR benchmark (OpenAI 2024).